**COOPS BIR - SAA**
Cooperative Society for Animal Husbandry & Feed Milling — Cameroon
Bafoussam / Douala — RCCM: RC/BFM/2026/B/452

# PROVENDA - SYSTEM MANAGEMENT

## EXHAUSTIVE FUNCTIONAL & TECHNICAL REFERENCE MANUAL (v2.5)

Feed Mill ERP, Multi-tender POS, Real-Time CMP WAC Valuation, Cloud Sync & Hardware Drivers

Offline-first architecture with permanent corporate branding & reverse chronological audit trail

| | |
|---|---|
| **Enterprise & Brand:** | COOPS BIR - SAA (Feed Mill & Livestock Supplies) |
| **Application Engine:** | Provenda ERP & POS Desktop / Web Edition v2.5 |
| **Accounting Currency:** | CFA Franc (FCFA / XAF) — Space-separated notation (e.g. 1 250 000 FCFA) |
| **Valuation Standard:** | Weighted Average Cost (WAC / CMP) incorporating landing freight fees |
| **Technical Stack:** | React 19, TypeScript, Vite, Tailwind CSS, IndexedDB, Supabase & Electron |
| **Hardware Support:** | 58 mm & 80 mm POS Thermal Receipt Printers (ESC/POS & Windows Spooler) |
| **Security & Control:** | Strict Role-Based Access Control (Developer, Admin, Manager, Cashier, Stockist) |
| **Edition Date:** | October 08, 2026 |

### TABLE OF CONTENTS & DOCUMENT OUTLINE

• Chapter 1: Global Architecture, Data Models & RBAC Hierarchy (Developer & Admin Separation)
• Chapter 2: Stock Engine, Packaging Unit Conversions & Dynamic WAC / CMP Calculation
• Chapter 3: Point of Sale Terminal (POS), Multi-tender Checkout & Controlled Discount Ceilings
• Chapter 4: Supply Chain, Goods Receptions, Landing Freight Allocation & Supplier Registry
• Chapter 5: Invoicing, Customer Credit Ledgers, Due Dates & Reversible Credit Notes (Avoirs)
• Chapter 6: Physical Inventory Sessions & Quantitative/Financial Variance Reconciliation
• Chapter 7: Thermal Receipt Printing Driver (58mm/80mm), Windows Spooler & Barcode Code 39
• Chapter 8: Cloud Supabase Synchronization, Offline-First Engine & JSON System Backups
• Chapter 9: Product Catalog & Dynamic Pricing Management ("Produits & Tarifs")
• Chapter 10: Developer System Console, Zero-Data Client Reset & Reverse Chronological Sorting
• Chapter 11: Financial Reports, CMP Stock Valuation & Immutable Security Audit Trail
• Chapter 12: Comprehensive Codebase Function Directory (AppContext, Services & Signatures)

COOPS BIR - SAA © 2026 — Provenda ERP & POS All-in-One Software System

# 1. Global Architecture & Role-Based Access Control (RBAC)

Provenda is architected around a unified transactional React Context state engine (AppContext), guaranteeing full atomic consistency across inventory ledgers, checkout registers, purchase receptions, customer debts and the immutable audit trail. In accordance with strict internal security standards, the system distinguishes five specific profiles: Developer, Administrator, Depot Manager, Cashier, and Stockist.

## 1.1 System Role Hierarchy & Exclusive Privileges

| Role Profile | Account Cardinality | Discount Limit | Key Scope & Permissions | Creation Authority |
|---|---|---|---|---|
| Developer | Exactly 1 (Protected) | 100% max | Full DB console, cloud sync setup, zero-data factory reset. Hidden from non-developers. | Can create Administrator & all other roles. |
| Administrator | Exactly 1 (Active) | 100% max | Company parameters, catalog pricing, full stock/purchase validation, financial reports. | Can create Manager, Cashier, Stockist (not Admin). |
| Depot Manager | 1 or more | 15% max | POS sales, purchase orders, goods receipt validation, stock adjustments, customer settlements. | Cannot create user accounts. |
| Cashier | 1 or more per shift | 0% (Strict) | POS counter sales, multi-tender receipts, cash drawer, credit debt collection. View only. | Cannot create user accounts. |
| Stockist / Warehouse | 1 or more | 0% (Blocked) | Physical goods reception (BL), stock counting, loss/damage adjustments with reason. No sales. | Cannot create user accounts. |

## 1.2 Core Data Models & Entities

| Entity | Identifier Format | Critical Attributes | Business & Accounting Purpose |
|---|---|---|---|
| Product | UUID / Code PRD-001 | baseUnit, saleUnit, conversionFactor, cmp, stockCurrentBaseUnit | Feed mill article managed in kg or bag with dynamically updated WAC/CMP. |
| Invoice | FAC-YYYY-XXXXXX | customerId, items[], totalNet, totalCostOfGoodsSold, grossMargin | Sales invoice with itemized profitability, taxes and payment allocations. |
| Purchase | ACH-YYYY-XXXXXX | supplierId, items[], transportFees, otherFees, status | Supply order incorporating landed freight fees into product inventory valuation. |
| StockMovement | MVT-YYYY-XXXXXX | productId, type, quantityBaseUnit, costUnitCMP, justification | Immutable ledger of every inbound, outbound and adjustment flow. |
| InventorySession | INV-YYYY-XXXXXX | theoreticalStock, physicalStock, varianceQty, varianceValue | Physical stocktake session reconciling discrepancies valorized at CMP. |
| Customer | CLI-001 | creditLimit, currentDebt, type (farmer, cooperative, reseller) | Client account with strictly enforced debt ceilings and maturity dates. |
| ReceiptPrinterConfig | Configuration object | paperWidth (58/80mm), printerName, autoPrint, showLogo | POS thermal printing driver settings for Windows spooler and desktop bridge. |
| AuditLog | AUD-YYYY-XXXXXX | userId, action, targetObject, oldValue, newValue, justification | ISO-compliant non-repudiation audit trail for all sensitive financial operations. |

# 2. Stock Engine & Weighted Average Cost (WAC / CMP) Calculation

## 2.1 Official Mathematical WAC Formula

**RECALCULATION OF WEIGHTED AVERAGE COST UPON GOODS RECEIPT:**

$$New_WAC = \left( (Initial_Stock \times Old_WAC) + Inbound_Cost \right) / \left( Initial_Stock + Received_Qty \right)$$

Where Inbound_Cost = (Received_Qty × Unit_Price) + Transport_Fees + Handling_Landing_Fees

## 2.2 Attribution of Inbound Landing & Freight Costs

Under OHADA (SYSCOHADA) and international inventory valuation standards for agricultural processing units, the acquisition cost of raw materials (Corn, Soya meal, Premixes) must include all direct expenditures necessary to bring the inventory to its present location and condition. When goods are received in Provenda, transport fees and offloading expenses are proportionally added to the product value, preventing any artificial inflation of margins.

## 2.3 Multi-Unit Packaging & Conversion Engine

| Commercial Packaging | Sale Unit | Stock Base Unit | Conversion Factor | Operational Execution in POS |
|---|---|---|---|---|
| 50 kg Bag (Finisher / Broiler feed) | bag (sac) | kilogram (kg) | 50 | Sale of 2 bags = Auto deduction of 100 kg in inventory at CMP. |
| 25 kg Bag (Starter chick feed) | bag (sac) | kilogram (kg) | 25 | Sale of 4 bags = Auto deduction of 100 kg in inventory at CMP. |
| Loose bulk retail by the kilo | kilogram (kg) | kilogram (kg) | 1 | Sale of 12.5 kg = Exact reduction of 12.5 kg in inventory. |
| 20 Liter Jug (Liquid vitamins / Molasses) | jug (bidon) | liter (L) | 20 | Receipt of 5 jugs = Auto increment of 100 L in warehouse. |
| 1 Ton Pallet (Bulk grain supplies) | pallet (palette) | kilogram (kg) | 1 000 | Receipt of 2 pallets = Stock increment of 2 000 kg. |

# 3. Point of Sale Terminal (POS) & Checkout Engine

The POS terminal is optimized for high-volume sales counters with instant barcode and product code lookup, dynamic packaging unit selector, live profitability margin indicators, multi-tender payment splitting (Cash, Orange Money, MTN MoMo, Bank Transfer, Customer Credit), and automatic thermal receipt generation.

## 3.1 Live Profitability & Gross Margin Formulas

```
1. Gross Subtotal = Sum ( Line Quantity × Catalog Unit Sale Price )
2. Net Invoice Revenue = Gross Subtotal - Total Approved Commercial Discount
3. Cost of Goods Sold (COGS) = Sum ( Base Quantity × Product WAC / CMP )
4. Gross Margin = Net Revenue - COGS   |   Margin Rate (%) = (Gross Margin / Net Revenue) × 100
```

## 3.2 Commercial Discount Authorization Matrix

| User Profile | Max Allowed Discount | Enforcement Mechanism | Audit & Exception Logging |
|---|---|---|---|
| Cashier | 0% (Strictly Blocked) | Input locked to 0%. Any discount field is rejected by the transaction engine. | Impossible to checkout with discount without manager override. |
| Depot Manager | Up to 15% maximum | Soft warning displayed above 10%. Hard block if discount exceeds 15%. | Manager identity, client code and discount justification logged on invoice. |
| Administrator | Up to 100% (Full exception) | Full override capability with mandatory reason prompt for auditing. | Logged in immutable audit trail with before/after financial impact. |
| Developer | 100% (Console) | Full override during testing and calibration mode. | Special developer maintenance flag recorded. |

## 4. Supply Chain, Goods Receptions & Supplier Ledger

Supply chain management ensures complete traceability of grain and feed input purchases. Purchase orders originate as drafts (ACH-YYYY-XXXXXX). Upon physical delivery at the warehouse, the goods receipt validation checks incoming quantities, allocates transport fees, automatically updates the stock base units, recalculates each item's WAC/CMP, and marks the purchase as received.

## 5. Invoicing, Customer Credit Ledgers & Credit Notes (Avoirs)

Every completed counter transaction generates an official sequentially numbered invoice (FAC-YYYY-XXXXXX). For partner farmers and cooperative members with approved credit facilities, the system monitors credit ceilings and due dates. In the event of order cancellation or dispute, an official Credit Note (AVO-YYYY-XXXXXX) is generated, reintegrating returned items into inventory and reversing customer receivables.

| Invoice Status | Debt Balance | Stock Impact | Allowed Operational Action |
|---|---|---|---|
| Paid (Payée) | 0 FCFA | Decremented at sale at WAC | Print duplicate receipt, view A4 invoice. |
| Credit (À crédit) | > 0 FCFA (Full) | Decremented at sale at WAC | Record partial/full debt payment, send payment reminder. |
| Partial (Partielle) | > 0 FCFA (Balance) | Decremented at sale at WAC | Receive secondary payments until balance reaches 0 FCFA. |
| Cancelled (Annulée) | 0 FCFA (Reversed) | Reintegrated via Credit Note | Historical viewing only. Stock returned to warehouse. |

## 6. Physical Inventory Sessions & Reconciliation at WAC

Inventory management supports scheduled and spot physical counting sessions (INV-YYYY-XXXXXX). The system freezes theoretical stock at session opening, compares it against field counts, and automatically computes quantitative variances (Surplus/Deficit) and financial variances valorized at current WAC/CMP. Validating the session creates balancing stock movements and records an audit log entry.

## 7. POS Thermal Receipt Printing Driver (58 mm & 80 mm)

Provenda includes a built-in POS thermal receipt printer driver supporting both 58 mm and 80 mm thermal paper formats. The driver connects with the Windows print spooler, USB and network receipt printers, and the native Electron desktop bridge. Every printed receipt includes the corporate logo, store header, tax credentials, itemized quantities and prices, payment methods, cash tender change calculation, and an official Code 39 barcode.

| Printer Capability | 58 mm Paper Width | 80 mm Standard POS Width | Operational Benefit |
|---|---|---|---|
| Printable Columns | 32 characters per line | 42-48 characters per line | Adapts cleanly to small mobile Bluetooth printers and heavy-duty desktop terminals. |
| Item Breakdown | 2-line compact stack per item | 4-column tabular alignment | Clear reading of quantities, unit prices, discounts and total line values. |
| Barcode Generation | High-density Code 39 | Standard Code 39 barcode | Allows instant ticket lookup and barcode scanning for returns and customer audits. |
| Reprint Duplicate Marking | *** DUPLICATA / RÉIMPRESSION *** | *** DUPLICATA / RÉIMPRESSION *** | Prevents cash theft and fraudulent duplicate re-printing. |

# 8. Cloud Supabase Synchronization & Offline Local-First Engine

Provenda functions with a 100% autonomous local-first architecture. All cash desk, inventory and sales operations execute instantly in local memory and IndexedDB without requiring an internet connection. When network connectivity is active, the background sync engine automatically transmits transactions to the cloud PostgreSQL database (Supabase) and pulls remote updates. Default credentials in supabaseDefaultConfig.ts provide zero-configuration backup, complemented by manual JSON file export.

# 9. Product Catalog & Dynamic Pricing Management ("Produits & Tarifs")

The dedicated 'Produits & Tarifs' module allows authorized managers and administrators to view, search, create and maintain feed articles and raw materials. For each item, users configure the commercial packaging (50 kg bag, 25 kg bag, liter jug, loose kg), stock conversion ratios, purchase costs, catalog sale prices and minimum safety alert thresholds. Every price revision is logged in the historical price ledger with date, previous price, new price, user ID, and justification.

| Catalog Category | Sample Products | Stock Management Unit | Price Revision Rule |
|---|---|---|---|
| Raw Materials (Matières Premières) | Corn, Soya meal, Wheat bran, Oyster shells | Kilogram (kg) / Ton | Frequent price fluctuations based on agricultural harvest seasons. |
| Finished Feed (Aliments Finis) | Broiler Starter, Broiler Finisher, Layer Feed, Pig Grower | Bag of 50 kg or 25 kg | Stable commercial price set to maintain targeted gross margin. |
| Concentrates & Premixes | Concentrate 5%, Broiler Premix, Vitamin Mineral Premix | Kilogram (kg) or 25 kg sack | High-value input with precise weight monitoring. |
| Supplies & Veterinary Additives | Liquid vitamins, Dewormers, Disinfectants, Molasses | Liter (L) / Unit | Packaged retail items with fixed catalog sale prices. |

# 10. Developer System Console, Zero-Data Reset & Sorting Standards

To guarantee a clean handover of the application to the final client, the Developer profile has an exclusive maintenance console. The 'Réinitialiser aux Données Initiales' function performs a complete purge of all operational and transactional data (receipts, invoices, purchases, stock movements, physical inventory sessions, expenses, credit notes) while preserving the calibrated product catalog, company settings, and default logo. Additionally, all operational tables across the system follow strict reverse chronological ordering (newest items first) and format currency with non-breaking space thousand separators.

# 11. Financial Reports, CMP Stock Valuation & Immutable Security Audit Trail

The reporting module provides real-time visibility into business performance: Stock Valuation at WAC/CMP (with high-contrast totals row for crystal-clear readability), Sales Ledger & Margin Analysis, Cash Flow & Expense Summaries, and the ISO-compliant Audit Trail. Every sensitive event (price changes, overrides, invoice cancellations, stock reconciliations) is logged with user timestamp, old value, new value, and mandatory reason.

| Official Report | Report Objective | Key Valuation Metric | Export Options |
|---|---|---|---|
| Stock & WAC Valuation | Instant audit of available warehouse stocks | Total stock value = Sum (Stock Base Qty × CMP) | Printable A4 PDF, Excel/CSV export, screen preview. |
| Sales & Gross Margins | Commercial profitability by period and product | Gross Margin = Net Revenue - COGS at CMP | Date range filtering, cashier breakdown, printable PDF. |
| Cash Flow & Expenses | Operating overhead tracking and net cash balance | Net Cash = Cash Receipts - Operating Expenses | Expense categorized breakdown, receipt vouchers. |
| Audit Trail (Piste d'Audit) | ISO forensic security and fraud prevention | Immutable log of all user interventions & overrides | Tamper-proof event log, developer/admin filters. |

# 12. Comprehensive Codebase Function Directory (AppContext & Services)

This technical directory specifies the core methods implemented in AppContext.tsx and surrounding services. It documents TypeScript signatures, inputs, outputs, exceptions, and execution logic for engineers and auditors.

## Authentication, RBAC & Developer Role

| Function | TypeScript Signature | Operational Logic & Algorithm |
|---|---|---|
| login(email, password) | (email: string, password: string) => { success: boolean; error?: string } | Authenticates user credentials, validates active account status, updates lastLogin timestamp, and creates user session. |
| logout() | () => void | Terminates active session, clears authentication state, and redirects user to secure login interface. |
| switchRole(role) | (role: Role) => void | Switches active role for testing privilege boundaries (developpeur, administrateur, responsable, caissier, magasinier). |
| addUser(user) | (user: Omit<User, "id"\|"createdAt">) => { success: boolean; error?: string } | Creates an employee user. If Developer, can create Administrator. If Administrator, can create Manager, Cashier, Stockist. |
| updateUser(user) | (user: User) => void | Updates employee credentials, status, discount limits, or role with full audit traceability. |
| deactivateUser(id) | (id: string) => void | Deactivates an employee account immediately revoking access while preserving past transactional history. |

## Product Catalog & Dynamic Pricing

| Function | TypeScript Signature | Operational Logic & Algorithm |
|---|---|---|
| addProduct(product) | (product: Omit<Product, "id"\|"createdAt"\|"updatedAt">) => { success: boolean; error?: string } | Creates new feed item or raw material with base units, packaging conversion factor, purchase price, catalog sale price and alert threshold. |
| updateProduct(product) | (product: Product) => { success: boolean; error?: string } | Updates product information, description, category, packaging and reorder thresholds with audit logging. |
| updateProductPrice(id, newPrice, reason) | (productId: string, newPrice: number, reason: string) => { success: boolean; error?: string } | Revises catalog sale price, logs change in priceHistory with previous price, author, date and mandatory reason. |
| deleteProduct(id) | (productId: string) => { success: boolean; error?: string } | Soft-archives product if no active transactions, preventing deletion of items with accounting history. |

## POS Sales, Checkout, Margins & Avoirs

| Function | TypeScript Signature | Operational Logic & Algorithm |
|---|---|---|
| createSale(...) | (customerId, items, discount?, payment?, notes?, isCredit?, creditDueDate?, spId?) => { success: boolean; invoiceId?: string; error?: string } | Executes atomic checkout transaction: validates stock, checks user discount ceiling, computes gross margin at WAC, generates FAC invoice, decrements stock, records payments and logs audit. |
| cancelInvoiceWithCreditNote(id, reason) | (invoiceId: string, reason: string) => { success: boolean; error?: string } | Cancels sale invoice: generates official Credit Note AVO-YYYY-XXXXXX, restocks inventory at WAC, reverses customer debt and records audit justification. |
| recordDebtPayment(...) | (invoiceId, amount, method, ref?, notes?) => { success: boolean; error?: string } | Records customer credit repayment: reduces invoice remaining balance, decrements total customer debt, and updates status to paid when balance reaches 0. |

## Supply Chain, Goods Reception & WAC Engine

| Function | TypeScript Signature | Operational Logic & Algorithm |
|---|---|---|
| createPurchaseDraft(p, spId) | (purchase, targetSalesPointId?) => string | Generates draft purchase order with reference ACH-YYYY-XXXXXX, line items, and estimated landing freight fees. |
| validatePurchaseReception(id) | (purchaseId: string) => void | Core WAC calculation engine: recalculates new WAC/CMP per item, increments base unit inventory, and closes purchase order. |
| cancelPurchase(id, reason) | (purchaseId: string, reason: string) => void | Cancels purchase order with mandatory justification note. Physical stock is unaffected. |

## Thermal POS Printing Driver (58mm/80mm)

| Function | TypeScript Signature | Operational Logic & Algorithm |
|---|---|---|
| printReceipt(invoice, settings, cfg, isReprint) | (invoice, settings, config, isReprint?) => Promise<{ success: boolean; error?: string }> | Dispatches formatted thermal ticket to physical printer via spooler, USB or isolated iframe printing. |
| formatReceiptThermal58mm(...) | (invoice, settings, config, isReprint?) => string | Generates high-contrast 32-column HTML ESC/POS document optimized for 58 mm mobile receipt printers. |
| formatReceiptThermal80mm(...) | (invoice, settings, config, isReprint?) => string | Generates standard 4-column 42-column HTML ESC/POS document for 80 mm POS counter printers. |
| savePrinterConfig(config) | (config: ReceiptPrinterConfig) => void | Persists thermal printer preferences (paper width, auto-print, logo display, |

| Function | TypeScript Signature | Operational Logic & Algorithm |
|---|---|---|
| | | printer name) to storage. |
| printTestReceipt(settings, config) | (settings, config) => Promise<{ success: boolean; error?: string }> | Sends a diagnostic test voucher to verify printer readiness, paper feed, alignment and barcode readability. |

## Cloud Supabase Sync & Zero-Data Client Reset

| Function | TypeScript Signature | Operational Logic & Algorithm |
|---|---|---|
| performAutomaticSync() | () => Promise<{ success: boolean; message?: string }> | Background engine: checks network, pushes local state changes to Supabase cloud, pulls remote updates. |
| pushAllLocalDataToSupabase() | () => Promise<{ success: boolean; message: string }> | Uploads all local tables (products, invoices, purchases, movements, customers, suppliers) to Supabase. |
| pullAllDataFromSupabase() | () => Promise<{ success: boolean; data?: any; message?: string }> | Downloads cloud state and restores local storage ensuring multi-device synchronization. |
| resetToDefaultData() | () => void | Developer exclusive maintenance action: purges all transactional entries (invoices, receipts, purchases, movements, sessions, expenses) while keeping product catalog, company settings and base64 logo. |
| exportSystemBackup() | () => string | Generates timestamped JSON backup file containing all application state tables for disaster recovery. |
| restoreSystemBackup(json) | (jsonString: string) => { success: boolean; message?: string } | Validates JSON structure and restores full application state with audit logging. |